TRANSIX

Algoritmusok

Készítették: Utasi Zalán Zoltán 13.I
Pálmai János Gábor 13.I

Tartalomjegyzék

1. Felhasználókezelés ... 3
1.1. Regisztráció (Register) ... 3
1.2. Bejelentkezés (Login) ... 3
1.3. Felhasználók és szerepkörök kezelése (adminisztrátor) ... 4
2. Törzsadatok és menetrend kezelése ... 6
2.1. Megálló és útvonal felvitele ... 6
2.2. Jármű és ülésterv felvitele ... 7
2.3. Menet (járat) létrehozása és sofőr hozzárendelése ... 7
2.4. Viteldíjsávok és bérlettípusok kezelése ... 8
3. Keresés és díjszámítás ... 10
3.1. Járatkeresés és menetrendböngészés ... 10
3.2. Útvonalhossz-meghatározás (GPS) ... 11
3.3. Viteldíj-számítás ... 11
3.4. Legkedvezőbb jegy-/bérlettípus meghatározása ... 12
3.5. Telítettségbecslés ... 13
4. Jegy- és bérletkezelés ... 14
4.1. Jegyvásárlás ... 14
4.2. Bérletvásárlás ... 15
4.3. QR-kód generálás és mentés ... 15
4.4. Saját jegyek, bérletek és foglalások megtekintése ... 16
4.5. Helyfoglalás módosítása és lemondása ... 17
4.6. Jegy lemondása és visszatérítés ... 17
5. Járművezetői alkalmazás ... 19
5.1. QR-érvényesítés (jegy és bérlet) ... 19
5.2. Utaslista megjelenítése ... 20
5.3. Menetrendtől való eltérés és rendkívüli esemény rögzítése ... 20
6. Valós idejű követés és értesítések ... 22
6.1. Pozíció fogadása és valós idejű járatkövetés ... 22
6.2. Értesítések (e-mail és alkalmazáson belüli) ... 23
7. Statisztikák és naplózás ... 24
7.1. Statisztikák és diszpécseri dashboard ... 24
7.2. Módosítások naplózása (auditLog) ... 24
8. Függelék: állapotértékek ... 26

1. Felhasználókezelés

1.1. Regisztráció (Register)

Cél: Új utas felhasználó létrehozása a rendszerben.

UI interakció:

1. A felhasználó a regisztrációs oldalon látja a mezőket: vezetéknév, keresztnév, email, telefonszám (nem kötelező), jelszó, jelszó megerősítése, valamint a Regisztráció gombot.
2. Kitölti a mezőket, majd rákattint a gombra.
3. Hibák esetén:
   - Üres vagy hibás formátumú mező → a mező mellett piros hibaüzenet
   - Túl rövid jelszó vagy a két jelszó nem egyezik → hibaüzenet a jelszó mező mellett
   - Már létező email → „Ez az email már foglalt"
4. Sikeres regisztráció esetén megjelenik a „Regisztráció sikeres!" üzenet.
5. A felhasználót átirányítjuk a bejelentkezési oldalra.

Bemenet: firstName, lastName, email, phone, password, passwordConfirm

Kimenet: user objektum (id, firstName, lastName, email, role), message

UI kimenet: sikerüzenet + átirányítás a loginra, vagy hibaüzenetek

Eljárás:

1. A rendszer ellenőrzi, hogy a kérés tartalmazza a kötelező mezőket (`firstName`, `lastName`, `email`, `password`, `passwordConfirm`).
2. Ha bármelyik hiányzik → hibaüzenet visszaadása (400).
3. Ellenőrzi az email formátumát, a mezők hosszát az adatbázis korlátai szerint (`email` max. 191, `firstName`/`lastName` max. 100, `phone` max. 30 karakter) és a jelszó minimális hosszát/erősségét.
4. Ellenőrzi, hogy a `password` és a `passwordConfirm` megegyezik.
5. Megvizsgálja a `users` táblában, hogy az `email` már foglalt-e (az `email` mező egyedi).
6. Ha foglalt → hibaüzenet visszaadása (409).
7. Ha minden valid:
   a) A jelszót hash-eli (pl. BCrypt) → `passwordHash`.
   b) Létrehoz egy új `users` rekordot: `role = passenger`, `discountCategory = none`, a `createdAt` és `updatedAt` mezőket az adatbázis tölti ki.
   c) Elmenti a rekordot (az `auditLog` trigger rögzíti a létrehozást, lásd 7.2).
8. Visszaadja az új felhasználó adatait (id, név, email, role) JSON formátumban, a jelszó hash nélkül (201).

Megjegyzés: *A járművezető, diszpécser és admin szerepkört csak adminisztrátor adhat ki (1.3), ezért a regisztráció mindig utas szerepkört hoz létre. A kedvezmény kategóriát (pl. diák) az admin állítja be igazolás alapján.*

1.2. Bejelentkezés (Login)

Cél: A felhasználó azonosítása és JWT hozzáférési token generálása.

UI interakció:

1. A felhasználó a bejelentkezési oldalon látja az email és jelszó mezőket, valamint a Bejelentkezés gombot.
2. Kitölti a mezőket, majd rákattint a gombra.
3. Hiba esetén a mezők felett hibaüzenet jelenik meg:
   - Üres mező → „Az email és a jelszó megadása kötelező"
   - Rossz adatok → „Érvénytelen email vagy jelszó"
4. Sikeres bejelentkezés esetén a rendszer a szerepkör alapján átirányít:
   - `passenger` → főoldal / saját jegyek, bérletek
   - `driver` → járművezetői érvényesítő felület
   - `dispatcher`, `admin` → dashboard
5. A token tárolása a kliensen: weben localStorage vagy HttpOnly cookie, mobilon biztonságos tároló (Flutter Secure Storage).
6. A token a további API-hívásoknál az `Authorization: Bearer <token>` fejlécben megy.

Bemenet: email, password

Kimenet: access_token, token_type, expires_in, user objektum (id, firstName, lastName, email, role)

UI kimenet: hibaüzenet, vagy átirányítás a szerepkörnek megfelelő felületre

Eljárás:

1. A rendszer ellenőrzi, hogy a kérés tartalmazza az `email` és `password` mezőket.
2. Ha bármelyik hiányzik → hibaüzenet (400).
3. Kikeresi a `users` táblából az adott `email`-hez tartozó felhasználót.
4. Ha nincs ilyen user → hibás hitelesítés (401). Ugyanazt az üzenetet adja, mint rossz jelszónál, hogy ne derüljön ki, melyik email létezik.
5. Ellenőrzi a jelszó egyezését a tárolt `passwordHash` és a beérkező jelszó között (BCrypt verify).
6. Ha nem egyezik → hibás hitelesítés (401).
7. Ha minden rendben van:
   a) Összeállítja a JWT claim-eket: `sub` (users.id), `email`, `role`, `exp` (lejárat, pl. 60 perc).
   b) Aláírja a tokent a szerver titkos kulcsával (HMAC SHA-256).
   c) Opcionálisan refresh tokent generál (külön tárolóban, pl. gyorsítótár, mert a modell nem tartalmaz ilyen táblát).
8. Visszaadja a tokent és a felhasználó azonosító adatait JSON formátumban (200).

Megjegyzés: *A titkos kulcs ne kerüljön GitHubra, hanem környezeti változóból vagy user-secrets-ből jöjjön. Ismétlődő sikertelen próbálkozásoknál érdemes IP- és emailalapú korlátozást (rate limit) alkalmazni.*

1.3. Felhasználók és szerepkörök kezelése (adminisztrátor)

Cél: Az adminisztrátor megtekintheti a felhasználókat, módosíthatja a szerepkörüket és a kedvezmény kategóriájukat (pl. járművezető kinevezése).

UI interakció:

1. Az admin a Felhasználók menüpontban rendezhető, kereshető listát lát (név, email, szerepkör, kedvezmény kategória, regisztráció ideje).
2. Kiválaszt egy felhasználót, és szerkesztő ablakban módosítja a szerepkört vagy a kedvezmény kategóriát.
3. A Mentés gombra kattintva megjelenik a „Módosítás elmentve" üzenet.
4. Hibák: saját admin jogosultság elvétele, az utolsó admin lefokozása, törlés kapcsolódó adatok miatt → hibaüzenet.

Bemenet: userId, role, discountCategory (a módosítandó mezők), keresési szűrők

Kimenet: frissített user objektum, message

UI kimenet: frissített lista és sikerüzenet, vagy hibaüzenet

Eljárás:

1. Ellenőrzi a JWT tokent és azt, hogy a hívó `role` értéke `admin`. Egyébként hiba (403).
2. Lista lekérdezésnél alkalmazza a keresést (név, email), a szűrést (`role`, `discountCategory`) és a lapozást a `users` táblán (a `role` mező indexelt).
3. Módosításnál ellenőrzi, hogy a felhasználó létezik, és a kapott `role`, `discountCategory` értékek érvényes enum értékek.
4. Ha a művelet az utolsó `admin` lefokozása vagy a saját jogosultság elvétele → elutasítja (409).
5. Ha a felhasználó `driver` szerepköre megszűnne, ellenőrzi, hogy nincs-e hozzá rendelt jövőbeli `trips` rekord (`driverId`). Ha van → hiba, előbb új sofőrt kell rendelni.
6. Elvégzi a módosítást (`UPDATE users`, `updatedAt` frissül). A változás az `auditLog` táblába kerül (a `changedBy` az admin azonosítója).
7. Törlésnél ellenőrzi a kapcsolatokat (`tickets`, `userPasses`, `notifications`, `trips`, `tripEvents`). Ha van kapcsolódó rekord, a törlést elutasítja, mert külső kulcsok védik az adatintegritást.
8. Visszaadja a frissített felhasználót JSON formátumban (200).

2. Törzsadatok és menetrend kezelése

2.1. Megálló és útvonal felvitele

Cél: A diszpécser/admin felvihet új megállókat és útvonalakat, és megadhatja az útvonal megállóinak sorrendjét, távolságát és menetidejét.

UI interakció:

1. A diszpécser a Megállók menüpontban az Új megálló gombra kattint, és megadja a nevet, a települést és a koordinátákat (vagy a térképen kijelöli a helyet).
2. Az Útvonalak menüpontban megadja az útvonalszámot és a nevet, majd hozzáadja a megállókat sorrendben, megadva az indulótól mért távolságot és menetidőt.
3. A Mentés gombra kattintva a rendszer visszajelzést ad.
4. Hibák: azonos útvonalszám, hibás koordináta, nem növekvő sorrend/távolság/menetidő → hibaüzenet a megfelelő mezőnél.

Bemenet: stop: name, city, latitude, longitude; route: routeNumber, name, isActive, stops[] (stopId, stopOrder, distanceFromStartKm, travelTimeMinutes)

Kimenet: létrehozott stop / route objektum a megállólistával

UI kimenet: sikerüzenet és az útvonal részletes nézete, vagy hibaüzenetek

Eljárás:

1. Ellenőrzi a jogosultságot (`dispatcher` vagy `admin`).
2. Megálló felvitelénél validálja a kötelező mezőket és a koordináták értéktartományát (szélesség −90…90, hosszúság −180…180). A pontosság a modell szerint `Decimal(10,8)` / `Decimal(11,8)`.
3. Figyelmeztet, ha azonos nevű és közeli koordinátájú megálló már létezik (duplikáció elkerülése, a `(latitude, longitude)` index segít).
4. Elmenti a `stops` rekordot.
5. Útvonal felvitelénél ellenőrzi, hogy a `routeNumber` egyedi.
6. Ellenőrzi a megállólistát:
   a) legalább két megálló szerepel,
   b) a `stopOrder` értékek 1-től folytonosan növekednek (egyedi index: `routeId`, `stopOrder`),
   c) az első megálló `distanceFromStartKm` és `travelTimeMinutes` értéke 0, a többi szigorúan növekvő,
   d) minden `stopId` létező megállóra mutat.
7. Ha a távolság nincs megadva, kiszámolja a szomszédos megállók közti távolságot Haversine-képlettel (lásd 3.2), és összegzi.
8. Adatbázis-tranzakcióban létrehozza a `routes` rekordot és a hozzá tartozó `routeStops` rekordokat. Hiba esetén visszagörgeti az egészet.
9. Módosításnál, ha az útvonalhoz már vannak jövőbeli `trips` rekordok, a megállók sorrendjének módosítása csak figyelmeztetéssel engedélyezett, mert már eladott jegyeket érinthet. Törlés helyett az `isActive = 0` érték használata ajánlott.

10. Visszaadja az elmentett adatokat (201).

2.2. Jármű és ülésterv felvitele

Cél: Járművek és a hozzájuk tartozó ülésterv (alaprajz) rögzítése, ami a helyfoglalás és a kapacitásszámítás alapja.

UI interakció:

1. A diszpécser a Járművek menüpontban az Új jármű gombra kattint, és megadja a rendszámot, a típust és az ülésrend paramétereit (sorok száma, oszlopok száma, kihagyott ülések).
2. A rendszer az alaprajz előnézetét mutatja, ahol egyes ülések megjelölhetők akadálymentesként.
3. Mentés után a jármű megjelenik a listában, szükség esetén inaktiválható.
4. Hibák: már létező rendszám, a megadott kapacitás nem egyezik az ülések számával → hibaüzenet.

Bemenet: licensePlate, model, capacity, isActive, rows, cols, kihagyott/akadálymentes ülések

Kimenet: vehicle objektum az ülések listájával

UI kimenet: alaprajz-előnézet és sikerüzenet, vagy hibaüzenet

Eljárás:

1. Ellenőrzi a jogosultságot (`dispatcher` vagy `admin`).
2. Validálja a mezőket, és azt, hogy a `licensePlate` egyedi.
3. Legenerálja az ülésterv elemeit a megadott sorok és oszlopok alapján: minden ülésre `seatNumber` (pl. 1A, 1B), `rowNum`, `colNum`.
4. Beállítja az `isWindow` értéket (az első és az utolsó oszlop ülései), az `isAccessible` értéket pedig a megjelölt ülésekre.
5. Ellenőrzi, hogy az ülések száma megegyezik a `capacity` értékével, és hogy a `(vehicleId, seatNumber)` párok egyediek.
6. Tranzakcióban létrehozza a `vehicles` rekordot és a hozzá tartozó `seats` rekordokat.
7. Jármű módosításánál, ha van jövőbeli menet, amelyhez eladott helyfoglalás tartozik, az ülésterv már nem módosítható (409). Kivonás esetén az `isActive = 0` érték használandó.
8. Visszaadja a jármű adatait és az ülések listáját (201).

2.3. Menet (járat) létrehozása és sofőr hozzárendelése

Cél: Egy útvonal konkrét indulásának (menetének) felvitele járművel és sofőrrel.

UI interakció:

1. A diszpécser a Menetrend menüpontban az Új menet gombra kattint.
2. Kiválasztja az útvonalat, a járművet, a sofőrt és az indulás idejét. Ismétlődő menethez megadja a napokat és a dátumtartományt.
3. A rendszer megjeleníti a számított érkezési időt.
4. Mentés után a menet megjelenik a menetrendben. Ütközés esetén (foglalt jármű vagy sofőr) hibaüzenet jelenik meg.

Bemenet: routeId, vehicleId, driverId, scheduledDeparture, (opcionális) ismétlődési szabály

Kimenet: trip objektum (vagy több, ismétlődés esetén)

UI kimenet: sikerüzenet és a menetrend frissítése, vagy ütközés-hibaüzenet

Eljárás:

1. Ellenőrzi a jogosultságot (`dispatcher` vagy `admin`).
2. Ellenőrzi, hogy az útvonal és a jármű aktív (`isActive = 1`), és hogy a `driverId` olyan felhasználóra mutat, akinek `role = driver`.
3. Kiszámolja a `scheduledArrival` értéket: `scheduledDeparture` + az útvonal utolsó megállójának `travelTimeMinutes` értéke (`routeStops`).
4. Ütközésvizsgálat: megkeresi, hogy a jármű vagy a sofőr foglalt-e egy átfedő időszakra (`trips` tábla, a `scheduledDeparture` és `scheduledArrival` alapján, kivéve a `cancelled` státuszú menetek). Ütközés esetén → hiba (409).
5. Létrehozza a `trips` rekordot: `status = scheduled`, `delayMinutes = 0`, a tényleges idő- és pozícióadatok üresek.
6. Ismétlődés esetén a fenti lépéseket minden érintett napra végrehajtja, egy tranzakcióban. Ha bármelyik nap ütközik, visszagörgeti, vagy jelzi az ütköző napokat.
7. Visszaadja a létrehozott menet(ek) adatait (201).

Megjegyzés: *A menetet törölni helyett lemondani érdemes (`status = cancelled`), mert már lehetnek hozzá eladott jegyek (lásd 4.6).*

2.4. Viteldíjsávok és bérlettípusok kezelése

Cél: A viteldíjszabás (távolságsávok) és az elérhető bérlettípusok karbantartása.

UI interakció:

1. Az admin a Díjszabás menüpontban táblázatban látja a viteldíjsávokat (alsó határ, felső határ, alapár) és a bérlettípusokat (név, időtartam, ár, célzott kedvezmény).
2. Új sávot vagy típust vehet fel, meglévőt szerkeszthet vagy inaktiválhat.
3. Hibák: átfedő vagy hézagos km-sávok, negatív ár, érvénytelen időtartam → hibaüzenet.

Bemenet: fareRates: minKm, maxKm, basePrice; passTypes: name, durationDays, price, targetDiscount, isActive

Kimenet: a mentett fareRates / passTypes rekord

UI kimenet: frissített táblázat és sikerüzenet, vagy hibaüzenet

Eljárás:

1. Ellenőrzi a jogosultságot (`admin`).
2. Viteldíjsáv mentésénél ellenőrzi, hogy `minKm < maxKm`, az ár nem negatív, és a `(minKm, maxKm)` pár egyedi.
3. Ellenőrzi, hogy az új sáv nem fedi át egyetlen meglévő sávot sem, és figyelmeztet, ha hézag keletkezik a sávok között (egy távolsághoz mindig tartoznia kell sávnak).

4. Bérlettípus mentésénél ellenőrzi, hogy a `durationDays` pozitív egész, a `price` nem negatív, a `targetDiscount` érvényes enum érték.
5. Elmenti a rekordot (`createdAt` az adatbázisban keletkezik). Az `auditLog` rögzíti a régi és az új értéket.
6. A már eladott bérletekre (`userPasses`) a változtatás nem hat vissza: a bérlettípus törlése helyett az `isActive = 0` beállítás szükséges, mert a `userPasses.passTypeId` külső kulcs hivatkozik rá.
7. Visszaadja a mentett rekordot (200/201).

3. Keresés és díjszámítás

3.1. Járatkeresés és menetrendböngészés

Cél: A felhasználó megtalálja a számára megfelelő járatokat indulási és érkezési megálló, dátum és további szűrők alapján.

UI interakció:

1. A főoldalon (és a járatkereső oldalon) a felhasználó látja a gyorskereső felületet: honnan, hová, dátum.
2. Megadja a megállókat (automatikus kiegészítéssel) és a dátumot, majd a Keresés gombra kattint.
3. Az eredménylista rendezhető (indulás, ár, utazási idő) és szűrhető (időszak, telítettség).
4. Egy járatra kattintva megjelennek a részletek: megállók, érkezési idők, ár, telítettségjelző.
5. Nincs találat esetén: „Nincs a keresésnek megfelelő járat", javaslattal a szomszédos napokra.
6. Érvénytelen adat (múltbeli dátum, azonos indulási és érkezési megálló) → hibaüzenet.

Bemenet: fromStopId, toStopId, date, szűrők (időszak, rendezés, oldalszám)

Kimenet: járatok listája (tripId, útvonalszám, indulás, érkezés, időtartam, távolság, ár, szabad helyek, telítettségi szint)

UI kimenet: találati lista vagy hibaüzenet / üres lista üzenet

Eljárás:

1. Ellenőrzi a kötelező paramétereket és azok érvényességét (a dátum nem múltbeli, a két megálló különbözik). Hiba esetén 400.
2. Megkeresi a `routeStops` táblában azokat az útvonalakat, amelyek érintik az indulási és az érkezési megállót úgy, hogy az indulási megálló `stopOrder` értéke kisebb.
3. Lekérdezi a `trips` táblából az ilyen útvonalak adott napi menetit, ahol a `status` értéke `scheduled` vagy `inProgress` (a `scheduledDeparture, status` index gyorsítja).
4. Kiszámolja minden meneten az indulási és érkezési időt a kiválasztott megállóknál: `scheduledDeparture` + a megálló `travelTimeMinutes` értéke + `delayMinutes`.
5. Kiszűri azokat a meneteket, amelyek az adott megállóból már elindultak.
6. Meghívja az útvonalhossz- és díjszámító modult a járat árához (3.2, 3.3 algoritmus).
7. Meghívja a telítettségbecslőt a járat szintjéhez (3.5 algoritmus).
8. Alkalmazza a szűrőket és a rendezést, majd lapozza az eredményt.
9. Visszaadja a találati listát JSON formátumban (200).

3.2. Útvonalhossz-meghatározás (GPS)

Cél: A megtett vagy tervezett útszakasz hosszának automatikus meghatározása a megállók adatai és GPS-koordinátái alapján.

UI interakció:

1. Az utas jegyvásárláskor kiválasztja a fel- és leszállási megállót, vagy a térképen jelöli azokat.
2. A rendszer megjeleníti a számított távolságot (km) a kiválasztás mellett.
3. Hiba esetén (a leszállási megálló az útvonalon az indulás előtt van, hiányzó koordináta) hibaüzenet jelenik meg.

Bemenet: routeId (vagy tripId), startStopId, endStopId

Kimenet: distanceKm (két tizedesre kerekítve)

UI kimenet: távolság megjelenítése, vagy hibaüzenet

Eljárás:

1. Betölti a két megálló `routeStops` rekordját az útvonalon.
2. Ellenőrzi, hogy mindkét megálló szerepel az útvonalon, és a felszállás `stopOrder` értéke kisebb a leszállásénál. Különben hiba (400).
3. Ha mindkét megállónál ki van töltve a `distanceFromStartKm`, a távolság a leszállási és a felszállási érték különbsége.
4. Ha hiányzik az adat, a köztes megállókat a `stopOrder` szerint sorba rendezi, és minden szomszédos megállópárra kiszámolja a távolságot a `stops.latitude` és `stops.longitude` alapján a Haversine-képlettel:
   a) $a = \sin^2(\Delta\varphi/2) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2(\Delta\lambda/2)$
   b) $d = 2 \cdot R \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{(1-a)})$, ahol $R = 6371$ km
5. Összegzi a szakaszok hosszát.
6. Az útvonal úthálózat szerinti hosszához korrekciós tényezőt alkalmaz (pl. 1,2), ha csak légvonalbeli adat áll rendelkezésre.
7. Visszaadja a távolságot km-ben.

Megjegyzés: *Az `distanceFromStartKm` mezőt útvonal-felvitelkor érdemes kitölteni (2.1), így futásidőben csak egy kivonás kell.*

3.3. Viteldíj-számítás

Cél: Az utazás távolsága és az utas kedvezmény kategóriája alapján az egyszeri jegy árának kiszámítása.

UI interakció:

1. Jegyvásárláskor a felhasználó látja az egyszeri jegy árát a kiválasztott szakaszra.
2. Kedvezményre jogosult felhasználónál (pl. diák) az alapár és a kedvezményes ár is látszik.
3. Hibás szakasz esetén az ár helyett hibaüzenet jelenik meg.

Bemenet: distanceKm, userId (a kedvezmény kategóriához)

Kimenet: basePrice, discountPercent, finalPrice

UI kimenet: ár és kedvezmény megjelenítése

Eljárás:

1. Meghívja az útvonalhossz-meghatározást (3.2), és megkapja a `distanceKm` értéket.
2. Megkeresi azt a `fareRates` sávot, amelyre `minKm ≤ distanceKm ≤ maxKm`. Ha nincs ilyen sáv → hiba (422), és naplózza a hiányzó díjszabást.
3. Az alapár a sáv `basePrice` értéke.
4. Lekéri a `users.discountCategory` értéket, és a konfigurációban megadott szabály szerint meghatározza a kedvezmény százalékát (`none` esetén 0).
5. Kiszámolja a végösszeget: `finalPrice = basePrice × (1 − discountPercent / 100)`.
6. A végösszeget a pénznemnek megfelelően kerekíti (pl. 5 Ft-ra), és két tizedesre formázza, ahogy az a `tickets.price` mezőben (`Decimal(10,2)`) tárolódik.
7. Visszaadja a számított értékeket.

Megjegyzés: *A kedvezmény százalékát nem tárolja külön tábla, ezért konfigurációban (appsettings) érdemes megadni.*

3.4. Legkedvezőbb jegy-/bérlettípus meghatározása

Cél: Az utas korábbi utazásai alapján annak a megállapítása, hogy egyszeri jegy vagy valamelyik bérlet (napi, heti, havi, éves) a legolcsóbb.

UI interakció:

1. Jegyvásárláskor, ha a rendszer bérletet találna kedvezőbbnek, ajánlat jelenik meg: „Havi bérlettel kb. X Ft-ot spórolhat".
2. A felhasználó az ajánlatra kattintva a bérletvásárlás oldalra jut, vagy elutasítja az ajánlatot, és egyszeri jegyet vesz.
3. Ha nincs elég előzmény vagy nincs megtakarítás, csak a jegy ára látható.

Bemenet: userId, aktuális szakasz díja (3.3)

Kimenet: ajánlott típus (jegy vagy `passTypeId`), becsült megtakarítás, magyarázat

UI kimenet: ajánlat megjelenítése a jegyvásárlási folyamatban

Eljárás:

1. Lekéri a felhasználó utazási előzményeit a `tickets` táblából (`status` értéke `active` vagy `used`) az elmúlt időszakra (pl. 60 nap), a `createdAt` és `price` mezők alapján.
2. Ha kevesebb mint a minimálisan szükséges számú utazás van (pl. 3), nem ad ajánlatot.
3. Betölti az aktív bérlettípusokat (`passTypes`, `isActive = 1`) olyan módon, hogy a `targetDiscount` megegyezzen a felhasználó `discountCategory` értékével vagy általános legyen.
4. Minden bérlettípusra (`durationDays` szerint, pl. 1, 7, 30, 365 nap):
   a) kiszámolja a megfigyelt időszak átlagos költését egy `durationDays` hosszú időszakra vetítve (a jegyek összára / megfigyelt napok × `durationDays`),
   b) hozzáadja az aktuálisan vásárolni kívánt jegy árát a legrövidebb időszakhoz,
   c) megtakarítás = vetített jegyköltség − bérlet `price`.
5. Kiválasztja a legnagyobb pozitív megtakarítást adó bérlettípust. Ha nincs pozitív megtakarítás, az egyszeri jegyet javasolja.

6. Ha a felhasználónak már van érvényes bérlete (`userPasses`, `status = active`, `validUntil` jövőbeli) ugyanarra a típusra, nem ajánl újat.
7. Visszaadja az ajánlatot indoklással JSON formátumban.

Megjegyzés: *Az ajánlás becslés. A felhasználó mindig szabadon dönthet.*

3.5. Telítettségbecslés

Cél: Az eladott jegyek és helyfoglalások alapján becslés adása a járat várható zsúfoltságáról.

UI interakció:

1. Jegyvásárláskor és a járatlistában az utas színes jelzést lát: zöld (kevés utas), sárga (közepes), piros (zsúfolt).
2. A diszpécser a dashboardon járatonként látja a telítettséget százalékban és a szakaszok szerinti bontást.
3. Telt házas járatnál a vásárlás gomb letiltott, és a felület másik járatot javasol.

Bemenet: tripId, (opcionális) startStopId, endStopId

Kimenet: occupancyPercent, szint (low / medium / high / full), szabad helyek száma

UI kimenet: színjelzés és szöveg az utasnak, részletes nézet a diszpécsernek

Eljárás:

1. Lekéri a járat kapacitását: `trips.vehicleId` → `vehicles.capacity`.
2. Betölti a járat érvényes jegyeit (`tickets`, `tripId`, `status` = `active` vagy `used`), valamint a `startStopId` és `endStopId` megállók `stopOrder` értékét a `routeStops` táblából.
3. Szakaszonként (két szomszédos megálló között) megszámolja azokat a jegyeket, amelyek a szakaszt érintik (`startOrder ≤ szakasz < endOrder`). A járat telítettsége a szakaszok maximuma.
4. Ha a lekérdezés konkrét szakaszra szól, csak az adott szakaszok maximumát nézi.
5. Kiszámolja a telítettséget: `foglaltság % = foglalt helyek / kapacitás × 100`.
6. Szintbe sorolja az értéket (pl. alacsony < 50%, közepes 50–80%, magas 80–100%, tele = 100%). A határok konfigurációban módosíthatók.
7. Opcionálisan korrigálja a becslést a korábbi, azonos napszakú menetek átlagos kihasználtságával (kereslet-előrejelzés).
8. Visszaadja az eredményt JSON formátumban.

Megjegyzés: *Az ülésfoglalás (`reservations`) menetre egyedi, ezért az ülés kiválasztása a teljes utat zárolja. A jegyszámlálás szakaszonként pontosabb képet ad a valós zsúfoltságról.*

4. Jegy- és bérletkezelés

4.1. Jegyvásárlás

Cél: Egyszeri jegy megvásárlása egy kiválasztott menetre és szakaszra, opcionális ülőhely-foglalással.

UI interakció:

1. A felhasználó a járat részleteinél a Jegyvásárlás gombra kattint.
2. Kiválasztja a fel- és leszállási megállót (alapértelmezés a keresésből), és ha kéri, az ülőhelyet az alaprajzon.
3. Látja a végösszeget, a telítettségjelzőt és az esetleges bérletajánlatot.
4. Megadja a fizetési adatokat, majd a Fizetés gombra kattint.
5. Hibák: elfogyott a hely, a járat már elindult, az ülőhely közben foglalttá vált, sikertelen fizetés → hibaüzenet.
6. Sikeres vásárlás után megjelenik a „Sikeres vásárlás!" üzenet és a jegy QR-kódja, majd átirányítás a saját jegyek oldalra.

Bemenet: userId (a tokenből), tripId, startStopId, endStopId, seatId (opcionális), paymentData

Kimenet: ticket objektum (id, ticketCode, price, status), reservation (ha van), message

UI kimenet: sikerüzenet, digitális jegy QR-kóddal, vagy hibaüzenet

Eljárás:

1. Ellenőrzi a bejövő adatokat: a menet létezik és `scheduled` vagy `inProgress` állapotú, a megállók az útvonalon vannak megfelelő sorrendben, és a menet az adott megállóból még nem indult el.
2. Meghívja a telítettségszámítást (3.5), és ellenőrzi, hogy van-e szabad hely a szakaszon. Ha nincs → hiba (409).
3. Kiszámolja az árat a 3.3 algoritmussal.
4. Ha ülőhelyet választott, ellenőrzi, hogy az ülés a menetben szereplő jármű ülése (`seats.vehicleId = trips.vehicleId`), és hogy a `(tripId, seatId)` pár még nem foglalt.
5. Feldolgozza a fizetést (fizetési szolgáltató vagy szimuláció). Bankkártyaadatot a rendszer nem tárol. Sikertelen fizetésnél hibát ad vissza (402), és nem hoz létre rekordot.
6. Sikeres fizetésnél adatbázis-tranzakcióban:
   a) létrehozza a `tickets` rekordot (`status = active`, `price`, `startStopId`, `endStopId`) a QR-azonosítóval (4.3),
   b) ha volt ülőválasztás, létrehozza a `reservations` rekordot (`ticketId`, `tripId`, `seatId`, `status = reserved`). Az egyedi index miatt ütközés esetén az egész tranzakció visszagörgetődik (409),
   c) létrehoz egy `notifications` rekordot (visszaigazolás).
7. Email visszaigazolást küld a jegy adataival.
8. Visszaadja a jegy adatait JSON formátumban (201).

4.2. Bérletvásárlás

Cél: Időszakos bérlet (napi, heti, havi, éves) megvásárlása és érvényességének rögzítése.

UI interakció:

1. A felhasználó a Bérletek oldalon látja az elérhető bérlettípusokat árakkal és érvényességgel.
2. Kiválasztja a bérlettípust és a kezdő dátumot.
3. Megadja a fizetési adatokat, majd a Vásárlás gombra kattint.
4. Hibák: érvénytelen kezdő dátum, már létező átfedő bérlet, nem jogosult kedvezményes típusra, sikertelen fizetés → hibaüzenet.
5. Siker esetén megjelenik a „Bérlet megvásárolva!" üzenet és a bérlet QR-kódja a saját bérletek oldalon.

Bemenet: userId (a tokenből), passTypeId, validFrom, paymentData

Kimenet: userPass objektum (id, passCode, validFrom, validUntil, status), message

UI kimenet: sikerüzenet és digitális bérlet, vagy hibaüzenet

Eljárás:

1. Ellenőrzi a bemenetet: a `passTypes` rekord létezik és `isActive = 1`, a `validFrom` nem múltbeli.
2. Ha a bérlettípus `targetDiscount` értéke nem általános, ellenőrzi, hogy megegyezik-e a felhasználó `discountCategory` értékével. Különben elutasítja (403).
3. Megvizsgálja, hogy a felhasználónak van-e már ugyanerre a típusra olyan aktív `userPasses` rekordja, amelynek érvényessége átfed az újjal. Ha van → hiba (409).
4. Kiszámolja a lejáratot: `validUntil = validFrom + durationDays`.
5. Feldolgozza a fizetést a `passTypes.price` összegre. Sikertelen fizetés → hiba (402).
6. Sikeres fizetés után tranzakcióban létrehoz egy `userPasses` rekordot: `status = active`, `passCode` (4.3).
7. Létrehoz egy `notifications` rekordot (visszaigazolás), és emailt küld.
8. A lejárat előtti emlékeztetőt az értesítési háttérfolyamat kezeli (6.2), mivel a `validUntil` mező alapján keresi a hamarosan lejáró bérleteket.
9. Visszaadja a bérlet adatait JSON formátumban (201).

4.3. QR-kód generálás és mentés

Cél: A jegyekhez és bérletekhez egyedi, hamisítás ellen védett azonosító (QR-kód) létrehozása és mentése az adatbázisba.

UI interakció:

1. A felhasználó a vásárlás után a saját jegyek/bérletek oldalon látja a QR-kódot.
2. A QR-kód a mobilalkalmazásban offline is megjeleníthető, ha egyszer már letöltődött.
3. Hiba esetén a felhasználó értesítést kap, és újratöltheti a kódot.

Bemenet: típus (ticket / pass), a kód létrehozásához szükséges rekordazonosító

Kimenet: ticketCode vagy passCode, qrPayload, QR-kép (Base64)

UI kimenet: a QR-kód képe a digitális jegyen/bérleten

Eljárás:

1. Generál egy véletlen, egyedi kódot (pl. 128 bites, hexadecimális formában, 32 karakter), amely illeszkedik a `ticketCode` / `passCode` mezők 64 karakteres korlátjába.
2. Ellenőrzi az egyediséget a `tickets.ticketCode` vagy a `userPasses.passCode` mezőben. Ütközés esetén újragenerál.
3. Elmenti a kódot a megfelelő rekordba (a vásárlási tranzakció részeként).
4. A QR-kód tartalma (payload): típusjelző (`T` vagy `P`), a kód, és egy HMAC SHA-256 aláírás első néhány karaktere, amelyet a szerver titkos kulcsával számol. Az aláírás tárolása nem szükséges, mert a kódból bármikor újraszámolható.
5. A payload alapján QR-kód képet készít (pl. QRCoder könyvtárral), amelyet kérésre generál, így a képet nem kell az adatbázisban tárolni.
6. Visszaadja a QR-kódot a hívónak.

Megjegyzés: *A QR-kód ne tartalmazzon személyes adatot, csak a kódot és az aláírást.*

4.4. Saját jegyek, bérletek és foglalások megtekintése

Cél: A bejelentkezett utas átlátható felületen éri el a jegyeit, bérleteit és helyfoglalásait.

UI interakció:

1. A felhasználó a Jegyeim oldalon füles nézetben látja a jegyeit (közelgő, felhasznált, lemondott), a bérleteit és értesítéseit.
2. A jegy kártyáján látható az útvonal, az indulás ideje, a megállók, az ülőhely és az állapot.
3. Egy kártyára kattintva megjelenik a QR-kód és a további műveletek (ülőhely módosítása, lemondás).
4. Ha nincs jegy, útmutató üzenet jelenik meg a járatkereső linkjével.

Bemenet: userId (a tokenből), szűrők (állapot, időszak, oldalszám)

Kimenet: tickets lista (útvonal, megállók, idők, ár, status, ülés), userPasses lista (típus, érvényesség, hátralévő napok)

UI kimenet: kártyák / táblázat a jegyekkel és bérletekkel

Eljárás:

1. A JWT tokenből kiolvassa a felhasználó azonosítóját. Más felhasználó adatai nem kérhetők le (403).
2. Lekérdezi a felhasználó `tickets` rekordjait, összekapcsolva a `trips`, `routes`, `stops` (start és end megálló), `reservations` és `seats` táblákkal.
3. Kiszámolja minden jegy megjelenítendő idejét a megállóknál (`scheduledDeparture` + `travelTimeMinutes` + `delayMinutes`).
4. Lekérdezi a `userPasses` rekordokat a `passTypes` táblával összekapcsolva, és kiszámolja a hátralévő napokat a `validUntil` alapján. Ha a `validUntil` elmúlt, de a `status` még `active`, a megjelenítésben lejártnak mutatja.

5. Alkalmazza az állapot- és időszakszűrőket, és lapoz.
6. A QR-kód a részletes nézetben kérésre generálódik (4.3).
7. Visszaadja az adatokat JSON formátumban (200).

4.5. Helyfoglalás módosítása és lemondása

Cél: Az utas az indulás előtt kiválaszthatja, módosíthatja vagy lemondhatja az ülőhelyét a jármű alaprajzán.

UI interakció:

1. A felhasználó a jegy részleteinél megnyitja a Helyfoglalás nézetet, ahol a jármű alaprajza látható.
2. A szabad, foglalt és a saját hely különböző színnel jelenik meg, az akadálymentes és ablak melletti ülések jelölve vannak.
3. Rákattint egy szabad ülőhelyre, majd a Foglalás / Módosítás gombra. Lemondásnál a Lemondás gombra kattint.
4. Hibák: a hely közben foglalttá vált, a módosítási határidő lejárt → üzenet és az alaprajz frissítése.
5. Siker esetén megjelenik a „Foglalás sikeres!" üzenet, és az új hely a jegyen is látszik.

Bemenet: ticketId, seatId, művelet (create / change / cancel)

Kimenet: reservation objektum (ticketId, tripId, seatId, status), message

UI kimenet: frissített alaprajz, sikerüzenet vagy hibaüzenet

Eljárás:

1. Lekéri a jegyet, ellenőrzi, hogy a hívóhoz tartozik, `status = active`, és a menet még nem indult el. A módosítási határidő (pl. indulás előtt 15 perc) konfigurációban állítható.
2. Lekéri az alaprajzot: a jármű összes ülése (`seats`, `vehicleId`) és a menet foglalt helyei (`reservations`, `tripId`).
3. Foglalás (a jegynek még nincs helye) esetén:
   a) ellenőrzi, hogy az ülés a menet járművéhez tartozik, és szabad,
   b) létrehozza a `reservations` rekordot (`status = reserved`). Az egyedi index (`tripId`, `seatId`) védi a dupla foglalástól, ütközésnél a rendszer hibát (409) ad vissza és frissíti az alaprajzot.
4. Módosítás esetén tranzakcióban frissíti a rekord `seatId` értékét az újra. Ha az új hely foglalt, a régi marad.
5. Lemondás esetén a rekord törlődik, így a hely újra foglalható. A régi adatot az `auditLog` megőrzi. A jegy érvényes marad, csak hely nélkül.
6. Frissíti a járat telítettségi adatait (3.5), ha szükséges.
7. Visszaadja a foglalás adatait JSON formátumban (200/201).

4.6. Jegy lemondása és visszatérítés

Cél: Az utas az indulás előtt lemondhatja a még fel nem használt jegyét, a rendszer pedig kiszámolja a visszatérítendő összeget.

UI interakció:

1. A felhasználó a jegy részleteinél a Jegy lemondása gombra kattint.
2. A rendszer megmutatja a visszatérítés összegét és a lemondás feltételeit, és megerősítést kér.
3. Megerősítés után megjelenik a „Jegy lemondva" üzenet és a jegy állapota frissül.
4. Hibák: a jegy már felhasznált, a menet elindult, a lemondási határidő lejárt → hibaüzenet.

Bemenet: ticketId (a tokenben szereplő felhasználóhoz tartozó)

Kimenet: frissített ticket, refundAmount, message

UI kimenet: megerősítő ablak és sikerüzenet, vagy hibaüzenet

Eljárás:

1. Lekéri a jegyet, és ellenőrzi, hogy a hívóhoz tartozik, és `status = active`.
2. Ellenőrzi, hogy a menet még nem indult el, és hogy a lemondás a megengedett határidőn belül történik.
3. Kiszámolja a visszatérítést a konfigurált szabály szerint (példa): ha az indulásig több mint 24 óra van, `refundAmount = price`. 24 és 2 óra között az ár 50%-a jár, ennél későbbi lemondásnál nincs visszatérítés. Ha a menet `cancelled`, mindig teljes visszatérítés jár.
4. Tranzakcióban beállítja a jegy `status` értékét `cancelled`-re, és törli a hozzá tartozó `reservations` rekordot, ha van.
5. A visszatérítést elindítja a fizetési szolgáltatónál (vagy szimulálja).
6. Létrehoz egy `notifications` rekordot, és emailt küld a visszatérítésről.
7. Az `auditLog` tartalmazza a művelet előtti és utáni állapotot.
8. Visszaadja az eredményt JSON formátumban (200).

5. Járművezetői alkalmazás

5.1. QR-érvényesítés (jegy és bérlet)

Cél: A járművezető vagy ellenőr gyorsan és megbízhatóan ellenőrzi az utasok jegyét vagy bérletét a QR-kód beolvasásával.

UI interakció:

1. A járművezető a nagy elemekkel kialakított érvényesítő képernyőn látja a kamera képét és az aktuális járat nevét.
2. A kamera beolvassa az utas QR-kódját.
3. Az eredmény egyértelmű, nagy méretű visszajelzésként jelenik meg: zöld (érvényes), piros (érvénytelen, lejárt, már felhasznált, más járat).
4. A hiba okát szöveg is jelzi, és hang- vagy rezgésjelzés kíséri.
5. Az érvényes jegyek az utaslistában is megjelennek.

Bemenet: qrPayload, tripId, driverId (a tokenből)

Kimenet: validationResult (valid / invalid), ok, jegy- vagy bérletadatok (utas neve, szakasz, ülés)

UI kimenet: zöld vagy piros eredménykijelzés és az utaslista frissítése

Eljárás:

1. Ellenőrzi a jogosultságot (`driver`), és azt, hogy a `trips.driverId` megegyezik a hívóval.
2. Dekódolja a QR-kód tartalmát, és újraszámolja az aláírást a szerver titkos kulcsával. Hibás aláírás → érvénytelen (hamis kód).
3. Jegy (`T`) esetén kikeresi a `tickets` rekordot a `ticketCode` alapján. Ha nem található → érvénytelen.
4. Ellenőrzi, hogy a jegy `tripId` értéke megegyezik az aktuális menettel. Más menet → érvénytelen.
5. Ellenőrzi az állapotot: `status = active` kell. A `used` (már felhasznált), `cancelled` vagy `expired` → érvénytelen, és az ok a hibaüzenetben megjelenik.
6. Érvényes jegynél atomikus módon beállítja: `UPDATE tickets SET status='used', validatedAt=NOW(), validatedByDriverId=@driverId WHERE id=@id AND status='active'`. Ha nem 1 sor módosult, közben más érvényesítette (duplikáció) → érvénytelen.
7. Bérlet (`P`) esetén kikeresi a `userPasses` rekordot a `passCode` alapján, ellenőrzi, hogy `status = active`, és a pillanatnyi idő a `validFrom` és `validUntil` között van. A bérlet nem „fogy el", így az állapota nem változik.
8. Visszaadja az eredményt és az utas megjelenítendő adatait JSON formátumban (200).
9. Hálózat nélkül az alkalmazás helyben ellenőrzi az aláírást és tárolja az érvényesítéseket, majd visszatéréskor szinkronizál. A közben történt duplikációkat a szerver a `status` ellenőrzésével kiszűri.

5.2. Utaslista megjelenítése

Cél: A járművezető megtekintheti az adott menet utasait, a helyüket és az érvényesítés állapotát.

UI interakció:

1. A járművezető a menet kiválasztása után az Utaslista fülön megnyitja a listát.
2. A lista soronként mutatja az utas nevét, a fel- és leszállási megállót, az ülőhelyet és az állapotot (érvényesítve / még nem).
3. Az összesítő sáv mutatja: eladott jegyek, érvényesítettek, hátralévők száma.
4. Megállóra szűrve látszik, kik szállnak fel vagy le az adott megállónál.

Bemenet: tripId, driverId (a tokenből), opcionális szűrő (megálló)

Kimenet: utaslista: ticketId, utas neve, startStop, endStop, seatNumber, status; összesítők

UI kimenet: táblázatos lista és összesítő sáv

Eljárás:

1. Ellenőrzi, hogy a hívó `driver`, és a `trips.driverId` megegyezik a hívó azonosítójával (vagy a hívó `dispatcher`/`admin`).
2. Lekérdezi a menet jegyeit (`tickets`, `tripId`, `status` = `active` vagy `used`).
3. Összekapcsolja a `users` (név), `stops` (start és end megálló), valamint a `reservations` és `seats` (ülés) táblákkal.
4. Megállószűrés esetén a `routeStops.stopOrder` alapján meghatározza a felszállókat és leszállókat.
5. Kiszámolja az összesítőket: összes jegy, `used` jegyek, `active` jegyek, foglalt ülések.
6. Rendezi a listát (pl. felszállási megálló, majd név).
7. Visszaadja az adatokat JSON formátumban (200).

Megjegyzés: *Bérletes utasok csak az érvényesítésük pillanatában látszanak, mert a bérlet nem kötődik konkrét menethez.*

5.3. Menetrendtől való eltérés és rendkívüli esemény rögzítése

Cél: A járművezető rögzíti a késést, műszaki hibát, útvonalváltozást vagy egyéb rendkívüli eseményt, amelyről a rendszer értesíti az utasokat és a diszpécsert.

UI interakció:

1. A járművezető az aktuális menet képernyőjén az Esemény rögzítése gombra kattint.
2. Kiválasztja az esemény típusát (késés, műszaki hiba, útvonalváltozás, egyéb), megadja a többletkésést percben és rövid leírást ír.
3. A Küldés gombra kattintva megjelenik a „Esemény rögzítve" üzenet.
4. Hibák: kötelező mező hiányzik, a menet nem a járművezetőé vagy nincs folyamatban → hibaüzenet.

Bemenet: tripId, driverId (a tokenből), eventType, delayMinutesAdded, description

Kimenet: tripEvent objektum, a menet frissített késése

UI kimenet: sikerüzenet és az esemény megjelenése a menet napló részében

Eljárás:

1. Ellenőrzi, hogy a hívó `driver`, a menet hozzá tartozik, és a `status` értéke `scheduled` vagy `inProgress`.
2. Validálja az `eventType` értéket, a `delayMinutesAdded` nem negatív egész szám, a leírás hossza elfogadható.
3. Tranzakcióban létrehozza a `tripEvents` rekordot (`tripId`, `driverId`, `eventType`, `delayMinutesAdded`, `description`).
4. Ha `delayMinutesAdded > 0`, növeli a `trips.delayMinutes` értékét ennyivel.
5. Ha az esemény menetlemondás (a diszpécserrel egyeztetve), a `trips.status` értéke `cancelled` lesz, és elindul a jegyek automatikus visszatérítése (4.6).
6. Ha a késés eléri a konfigurált küszöböt (pl. 10 perc), vagy útvonalváltozás történt, értesítést indít az érintett utasoknak (6.2).
7. A diszpécser dashboardján az esemény azonnal megjelenik.
8. Visszaadja a rögzített eseményt JSON formátumban (201).

6. Valós idejű követés és értesítések

6.1. Pozíció fogadása és valós idejű járatkövetés

Cél: A járművek aktuális helyzetének rögzítése és megjelenítése az utasoknak és a diszpécsernek, a késés és a várható érkezési idő számításával.

UI interakció:

1. Az utas a mobilalkalmazásban a Járatkövetés képernyőn térképen látja a járatot, a következő megállót és a várható érkezési időt.
2. A jármű ikonja néhány másodpercenként frissül.
3. Késés esetén a rendszer jelzi a késés mértékét. A diszpécser a dashboardon az összes aktív járatot látja.
4. Ha a pozíció nem frissül (pl. jelvesztés), az alkalmazás jelzi, hogy az adat nem friss.

Bemenet: (adóoldal) tripId, latitude, longitude, timestamp; (olvasóoldal) tripId

Kimenet: aktuális pozíció, következő megálló, becsült érkezési idő, késés percben

UI kimenet: jármű mozgása a térképen, ETA és késés kijelzése

Eljárás:

1. A járművezetői alkalmazás meghatározott időközönként (pl. 5–10 mp) elküldi a GPS-pozíciót az API-nak.
2. A szerver ellenőrzi a hitelesítést és azt, hogy a `trips.driverId` megegyezik a hívóval, valamint hogy a menet `scheduled` vagy `inProgress`.
3. Validálja az adatokat (koordináták értéktartománya, értelmes időbélyeg). Hibás adat elvetve.
4. Az első pozíciónál, vagy az Indulás gomb megnyomásakor beállítja az `actualDeparture` értéket, és a `status` értékét `inProgress`-re.
5. Elmenti az aktuális pozíciót a `trips.currentLatitude` és `trips.currentLongitude` mezőbe.
6. Haversine-távolsággal (3.2) meghatározza, hogy a jármű egy megálló közelében van-e (pl. 150 m). Megállóhoz érkezéskor kiszámolja a késést: `delayMinutes = tényleges idő − (scheduledDeparture + travelTimeMinutes)` az adott megállóra, percre kerekítve, legalább 0 értékre.
7. Ha a késés változott, frissíti a `trips.delayMinutes` mezőt. Ha átlépi a küszöböt, értesítést indít (6.2).
8. Az utolsó megállóhoz érkezéskor beállítja az `actualArrival` értéket, és a `status` értéke `completed` lesz.
9. Olvasóoldalon kiszámolja a várható érkezést a következő megállókra: `scheduledDeparture + travelTimeMinutes (routeStops) + delayMinutes`.
10. Az adatokat az ügyfél rövid időközönként lekérdezi (polling), vagy a szerver valós időben küldi (SignalR/WebSocket).

Megjegyzés: *A modell csak az aktuális pozíciót tárolja, a mozgás története nincs elmentve. Ha később szükség lenne rá, külön táblát érdemes felvenni.*

6.2. Értesítések (e-mail és alkalmazáson belüli)

Cél: Az utasok időben értesülnek a járat indulásáról, késéséről, útvonalváltozásáról, a jegy vagy bérlet vásárlásáról és a bérlet lejáratáról.

UI interakció:

1. A felhasználó e-mailben és a webes/mobil felület értesítési listájában (harang ikon, olvasatlan jelzéssel) kapja az értesítéseket.
2. Az értesítésre kattintva megnyílik a kapcsolódó jegy vagy járat részletei, az értesítés olvasottnak jelölődik.
3. A Beállítások oldalon az értesítéstípusok kapcsolhatók.

Bemenet: esemény típusa (reminder / delay / routeChange / expiry / info), érintett menet vagy bérlet

Kimenet: létrehozott `notifications` rekordok, elküldött e-mailek

UI kimenet: e-mail az utasnak, olvasatlan értesítés a felületen

Eljárás:

1. Az eseményt kiváltja egy háttérfeladat (időzítő, pl. hosted service) vagy egy másik modul (késés észlelése, esemény rögzítése, menetlemondás).
2. Időzített feladat (pl. percenként): megkeresi a közelgő meneteket (pl. 60 perc múlva indulnak: `trips.scheduledDeparture`), a hozzájuk tartozó aktív jegyeket, valamint a hamarosan lejáró bérleteket (`userPasses.validUntil`, pl. 3 napon belül).
3. Késés- vagy útvonalváltozás-esemény esetén az érintett menet `active` állapotú jegyeinek tulajdonosait kérdezi le.
4. Ellenőrzi az utas értesítési beállításait. Ha az adott típust kikapcsolta, kihagyja.
5. Ellenőrzi a `notifications` táblában, hogy ugyanaz az értesítés nem ment-e már ki a felhasználónak (duplikáció elkerülése).
6. Összeállítja az üzenet tartalmát sablon alapján (menet, idők, ok, teendő), és létrehozza a `notifications` rekordot (`userId`, `title`, `message`, `type`, `isRead = 0`).
7. Elküldi az e-mailt SMTP-n keresztül. Hiba esetén később újrapróbálja (pl. 3-szor).
8. A felhasználó az értesítést megnyitva (PATCH kérés) olvasottnak jelöli: `isRead = 1`. Az olvasatlan darabszámot a `(userId, isRead)` index gyorsítja.

7. Statisztikák és naplózás

7.1. Statisztikák és diszpécseri dashboard

Cél: Az eladási és beléptetési adatokból statisztikák, rendezhető táblázatok és diagramok előállítása a diszpécser és az admin számára.

UI interakció:

1. A diszpécser/admin a dashboardon látja a fő mutatókat: napi bevétel, eladott jegyek, aktív járatok, késések, telítettség.
2. Szűrőket állíthat be: időszak, útvonal, menet, jegy állapota, kedvezmény kategória.
3. Az adatok rendezhető táblázatban és diagramokon (vonal-, oszlop-, kördiagram) jelennek meg.
4. Az adatok exportálhatók (CSV/PDF).
5. Jogosulatlan felhasználónak hibaüzenet jelenik meg.

Bemenet: szűrők (dateFrom, dateTo, routeId, tripId, ticketStatus, groupBy, sortBy, sortDir, page)

Kimenet: összesített mutatók, táblázatos adatsorok, diagramadatok

UI kimenet: dashboard kártyák, táblázat és diagramok

Eljárás:

1. Ellenőrzi a JWT tokent és a szerepkört (`dispatcher` vagy `admin`). Egyébként 403.
2. Validálja a szűrőket (dátumtartomány, értelmes rendezési mező).
3. Összesítő lekérdezéseket futtat az adatbázison:
   a) bevétel: a `tickets.price` összege (`status` = `active` vagy `used`) és a `passTypes.price` összege a `userPasses` alapján, csoportosítva nap/hét/hónap szerint,
   b) eladott és beléptetett (`validatedAt` kitöltött) jegyek száma menetenként és útvonalanként,
   c) legnépszerűbb útvonalak, megállók és időpontok, átlagos telítettség (3.5),
   d) késések átlaga és maximuma (`trips.delayMinutes`), lemondott menetek száma (`status = cancelled`),
   e) rendkívüli események száma típusonként (`tripEvents`).
4. Alkalmazza a szűrést, rendezést és lapozást a táblázatos nézethez.
5. A diagramokhoz idősoros vagy kategóriák szerinti adatsorokat készít.
6. Visszaadja az eredményt JSON formátumban (200). Export esetén CSV vagy PDF fájlt generál.
7. A gyakran használt lekérdezéseket gyorsítótárazza, hogy a dashboard gyors maradjon.

7.2. Módosítások naplózása (auditLog)

Cél: Az adatbázis fontos tábláinak minden beszúrását, módosítását és törlését automatikusan rögzíteni a visszakövethetőség és az adatintegritás érdekében.

UI interakció:

1. Az admin a Napló menüpontban rendezhető, szűrhető táblázatban látja a bejegyzéseket (idő, tábla, rekord, művelet, felhasználó).
2. Egy sorra kattintva két oszlopban látja a régi és az új adatot, a változott mezők kiemelésével.
3. Szűrhető tábla, rekordazonosító, felhasználó és időszak szerint.

Bemenet: szűrők: tableName, recordKey, changedBy, időszak (changedAt)

Kimenet: auditLog bejegyzések listája (operationType, oldData, newData, changedBy, changedAt)

UI kimenet: napló táblázat és a változás részletei

Eljárás:

1. A nyomon követett táblákra (pl. `users`, `tickets`, `reservations`, `trips`, `userPasses`, `fareRates`, `passTypes`) adatbázis-triggerek (AFTER INSERT / UPDATE / DELETE) készülnek.
2. A backend minden adatbázis-kapcsolatnál beállítja a hívó felhasználó azonosítóját munkamenet-változóban (pl. `SET @currentUserId = ...`), így a trigger ki tudja olvasni.
3. Beszúrásnál a trigger egy `auditLog` rekordot hoz létre: `operationType = INSERT`, `newData` (JSON) a beszúrt sorral.
4. Módosításnál: `operationType = UPDATE`, az `oldData` és `newData` a változás előtti és utáni sort tartalmazza.
5. Törlésnél: `operationType = DELETE`, az `oldData` a törölt sort tartalmazza.
6. A trigger kitölti: `tableName`, `recordKey` (a rekord azonosítója, összetett kulcsnál összefűzve), `changedBy` (a munkamenet-változóból), `changedAt` (`NOW(6)`).
7. Érzékeny adatot (pl. `users.passwordHash`) a trigger nem másol a naplóba, vagy maszkolja.
8. Lekérdezésnél a szűrőket a `idxAuditTableRecord`, `idxAuditChangedAt` és `idxAuditChangedBy` indexek gyorsítják.
9. Csak `admin` olvashatja a naplót. A napló rekordjai nem módosíthatók és nem törölhetők az alkalmazásból.
10. Visszaadja a bejegyzéseket JSON formátumban (200).

8. Függelék: állapotértékek

Az adatbázismodell az enum típusú mezők értékeit nem sorolja fel. Az algoritmusok az alábbi értékeket használják feltételezésként. A csapat a saját megvalósítása szerint igazíthatja őket.

- users.role: `passenger`, `driver`, `dispatcher`, `admin`
- users.discountCategory és passTypes.targetDiscount: `none`, `student`, `senior` (a tényleges kategóriák a konfigurációtól függnek)
- trips.status: `scheduled`, `inProgress`, `completed`, `cancelled`
- tickets.status: `active`, `used`, `cancelled`, `expired`
- reservations.status: `reserved`, `cancelled`
- userPasses.status: `active`, `expired`, `cancelled`
- tripEvents.eventType: `delay`, `breakdown`, `routeChange`, `other`
- notifications.type: `reminder`, `delay`, `routeChange`, `expiry`, `info`
- auditLog.operationType: `INSERT`, `UPDATE`, `DELETE`